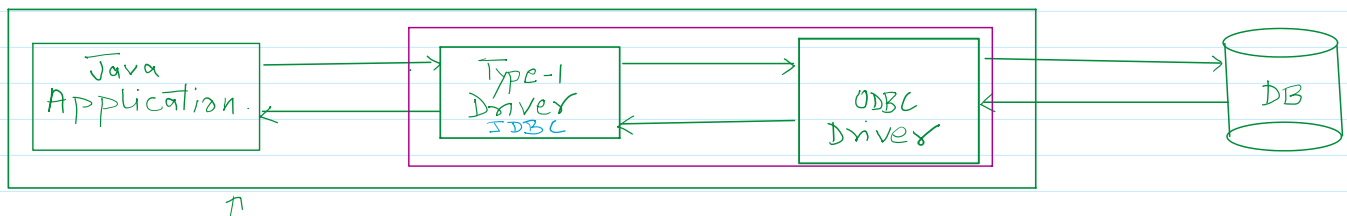


Types of JDBC Drivers

1. Type-1 Driver (JDBC-ODBC Bridge Driver or Bridge driver).
2. Type-2 Driver (Partly Java or Native-API - Partly Java driver or Native driver).
3. Type-3 Driver (All Java Net protocol Driver or Network protocol Driver or Middleware driver).
4. Type-4 Driver (Pure Java Driver, (or) Native Protocol driver (or) Thin Driver.)
5. Type-5 Driver (Progress Direct). (Not free, Proprietary software)
↑ not industry standard driver

Type-1 Architecture.



Must be present in client machine.

JDBC-ODBC Bridge OR Bridge driver.

- (*) Type-1 driver converts JDBC calls to ODBC call and ODBC calls to JDBC calls.
- (*) Internally this driver will take support of ODBC driver to communicate with database.
- (*) This driver is provided by SUN-MICRO SYSTEMS as the part of JDK.
- (*) Type-1 driver converts JDBC calls into ODBC calls and ODBC driver converts ODBC calls into database specific calls.
- (*) Hence Type-1 driver act as bridge between Java application and database.

Advantages:

1. It is very easy to use & maintain.
2. We are not required to install because it is available as the part of JDK.

1. This is very easy to use & maintain.

2. We are not required to install because it is available as the part of JDK.

3. Type-1 driver won't communicate with database directly. Hence, it is database independent driver, because of this migrating from one database to another will become easy.

Limitations:

1. It is the slowest driver among all drivers (Snail driver), because first it will convert JDBC calls to ODBC calls and ODBC driver converts ODBC calls into database specific calls.

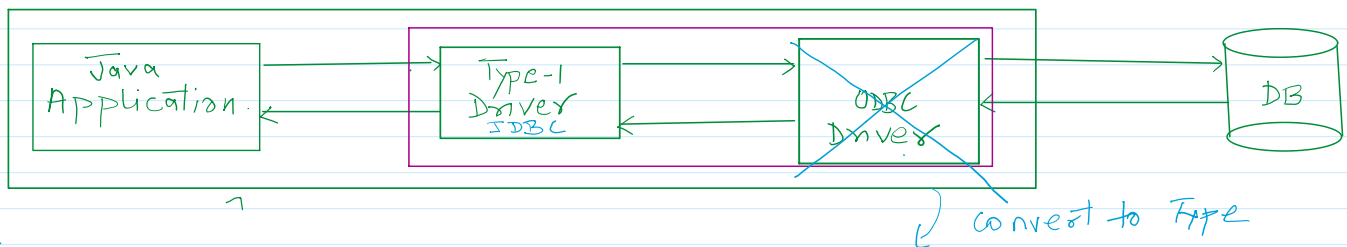
2. This driver internally depends on ODBC driver, which works flawlessly in windows O/S only. For other systems we need to install it. Hence, Type-1 driver is platform dependent driver.

3. No support from JDK 1.8 version onwards.

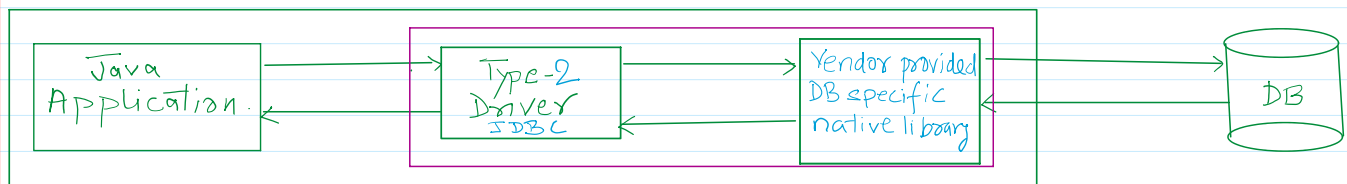
Note: Because of above limitations it is not recommended to use Type-1 driver.

Type-2 Driver

Type-1 Architecture.



Type-2 Architecture.



client-machine

Partly Java Native API driver

(*) It is also known as Native API partly Java driver.

(*) Type-2 driver is exactly same as Type-1 driver except that ODBC driver is replaced with Database vendor specific native library.

(*) Native libraries means the set of function written in non Java (mostly in C/C++).

- ⊗ Native libraries means the set of function written in non Java (mostly in C/C++).
- ⊗ We have to install vendor provided native libraries on the client machine.
- ⊗ Type-2 driver converts JDBC call into vendor specific native library call which can be understandable directly by database engine.

Advantages:

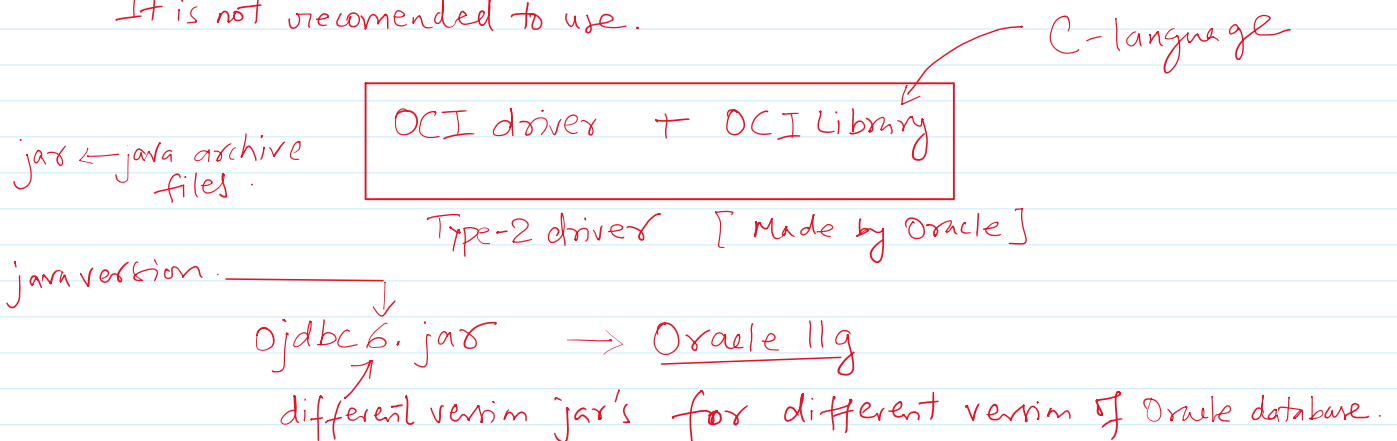
- ① When compared with Type-1 driver performance is high, because it requires only one level of conversion from JDBC to native library calls.
- ② No need of arranging ODBC driver.
- ③ When compare with Type-1 driver, portability is more, b'z Type-1 driver is more compatible in Windows, than any other O/S.

Limitations:

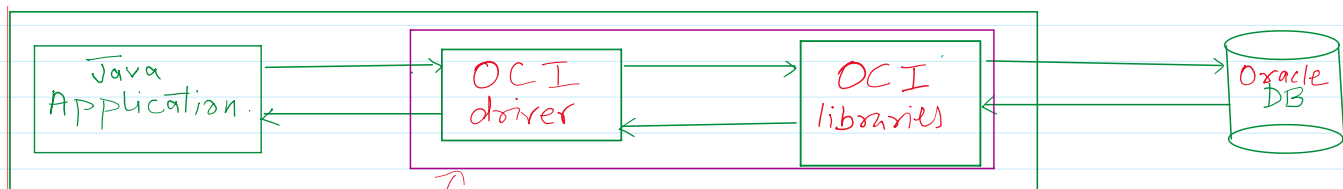
1. It is DB dependent driver, b'z it internally uses database specific native libraries. Hence, migrating from one db to another db will become difficult.
2. It is platform dependent Driver.
3. We required to install native libraries on the client machine.
4. There is no guarantee every db vendor will provide these drivers.

[oracle people will provide OCI drivers, but MySQL people won't]

Note: The only driver which is both platform and database dependent is Type-2.
It is not recommended to use.



Type-2 Architecture.



client machine

Passes Java Native API driver

ojdbc17.jar → Oracle 26ai database.

OCI (Oracle call interface): Type-2 driver is provided by Oracle.

OCI driver internally uses OCI libraries to communicate with DB.

OCI libraries contains C-language functions.

OCI driver and corresponding OCI libraries are available in the following jar file in the class path

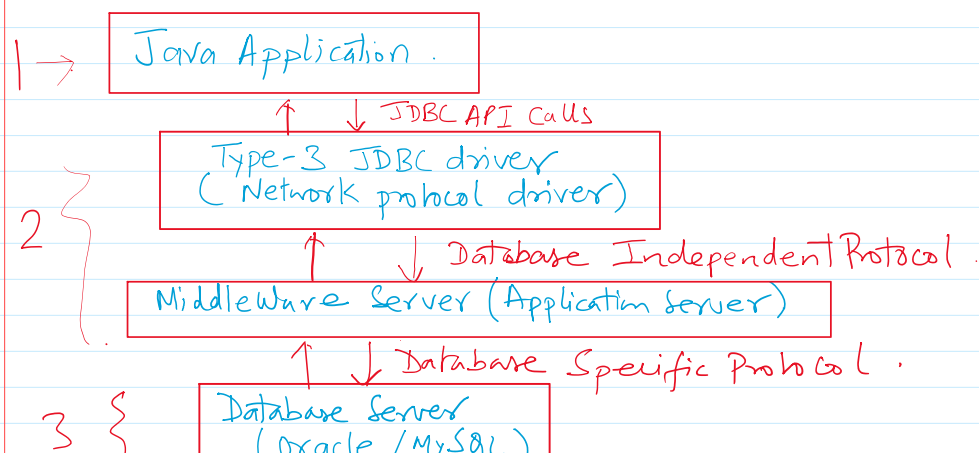
ojdbc17.jar → Oracle 26ai (internally uses java 17... java 21)

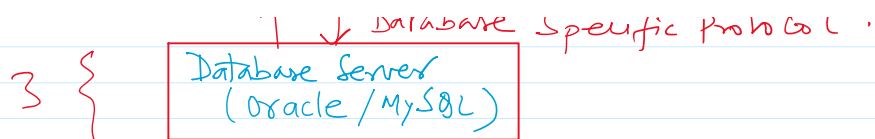
Type-3 JDBC Driver Architecture (Network Protocol Driver)

It converts JDBC calls into a database independent network protocol, which is then translated into the database-specific protocol by a middleware server.

This driver uses a 3-tier architecture.

1. Java Application.
2. Middleware/Application server
3. Database Server.





Working of Type-3 JDBC driver.

1. The Java application sends JDBC calls
2. The Type-3 JDBC driver converts JDBC calls into a database-independent network protocol.
3. These requests are sent to a middleware server.
4. The middleware server translates the request into the database specific protocol.
5. The database process the request and returns the result.
6. The middleware server sends the response back to the client application.

Important Features

- (*) Fully written in Java
- (*) Platform independent
- (*) Uses middleware server
- (*) No-database-specific libraries required on client side.
- (*) Support multiple databases.

Example of Type-3 JDBC Driver.

DataDirect Sequelink.

BEA Weblogic Driver

IBM middleware JDBC drivers.

Advantages of Type-3 JDBC Driver

1. Platform independent: Since the driver is written completely in Java, it can run on any platform.
2. No native Library Required: No database client libraries need to be installed on the client machine.
3. Support Multiple Databases: A single middleware server can connect to multiple databases.

3. Support Multiple Databases : A single middleware server can connect to different databases like :
 (X) Oracle (X) MySQL (X) PostgreSQL
 (X) SQL server.
4. Better Security : The middleware server can provide :
 (X) Authentication. (X) Authorization.
 (X) Encryption. (X) Connection Filtering.
5. Centralized Management : Database access logic is controlled from Middleware server.
6. Suitable for Internet Application : Efficient for web-based enterprise applications.

Limitations / Disadvantages of Type-3 JDBC Driver

1. Requires Middleware Server : An extra middleware server is necessary, increasing complexity.
2. Slower performance : Communication passes through an additional layer, causing overhead.
3. Expensive Maintenance : Middleware server requires configuration and maintenance.
4. More Network Traffic : Extra communication between client and middleware increases traffic.
5. Complex architecture : More difficult to configure Type-4 drivers.

Applications of Type-3 JDBC drivers.

Used in :

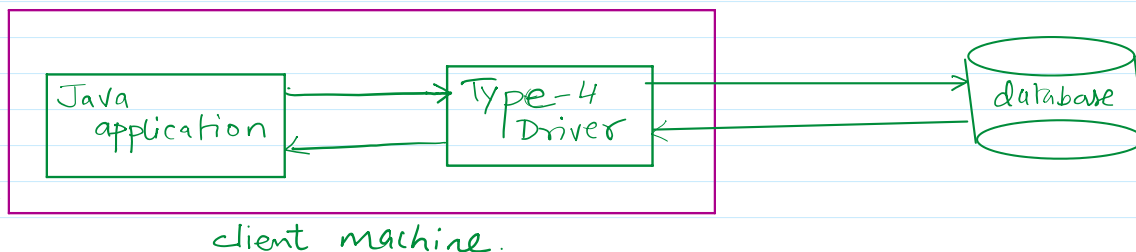
- (X) Enterprise applications (X) Banking systems (X) Web applications.
- (X) Distributed systems. (X) Cloud-based middleware systems.

In short we can say :

A type-3 JDBC driver converts JDBC calls into a database-independent protocol and sends them to a middleware server, which translates them into a database-specific protocol. It is platform independent and suitable for enterprise applications but require an additional middleware layer, making it slower and more complex than Type-4 drivers.

The Type-3 JDBC driver is a middleware based JDBC driver that provides flexibility, security and database independence. Although it introduces additional complexity and slower performance due to the middleware layer. It is useful for enterprise level distributed applications where centralized database management is required.

Type-4 driver



- (*) Type-4 driver is also known as pure Java Driver Or All Java Native Protocol Driver or Thin driver.
- (*) This Driver uses Database specific native protocol to communicate with database.
- (*) This Driver converts JDBC call into Database calls directly.
- (*) This Driver developed only in Java and hence it is also known as Pure Java Driver. Because of this it is platform independent driver.
- (*) This Driver won't require any ODBC Driver or native libraries at client side hence it is called thin driver.

Advantages.

- (*) Performance is more because conversion is only one step.
- (*) It is platform independent driver
- (*) It won't require any ODBC Driver or native libraries at client side or middleware server is also not needed.
- (*) It uses Database vendor specific native protocol to communicate with database and hence security is more.

Limitations

- (*) The only limitation of this Driver is it is a database dependent driver because it communicates with database directly.
e.g. Thin Driver for Oracle. }
connector/J Driver for MySQL. }